**TUGAS KEHADIRAN MINGGU KE-9**

**REKAYASA DAN ORGANISASI SISTEM BIG DATA**

---

*Senin, 20 April 2026*

Disusun oleh Kelompok 3:

| No | Nama | NIM |
|---|---|---|
| 1 | Olivia Intan Puspita | 2311110006 |
| 2 | Afifah Faiqatuzzahra | 2311110043 |
| 3 | Yasmine Alifta | 2311110044 |

## STUDI KASUS: SISTEM FILM

### 1. SOAL 1: LATAR BELAKANG & DESKRIPSI STUDI KASUS

Industri perfilman modern menghasilkan data yang sangat masif dan memiliki karakteristik yang sangat beragam. Data tersebut mencakup metadata film (seperti judul, genre, tahun rilis, rating, dan sinopsis) hingga hubungan sosial dan struktural yang kompleks (seperti kolaborasi antar aktor, rekam jejak sutradara, dan tingkat kemiripan antar film). Dalam ekosistem *Big Data*, satu model *database* tunggal seringkali tidak cukup optimal untuk menangani seluruh kebutuhan operasional tersebut secara bersamaan. Oleh karena itu, studi kasus ini merancang basis data Sistem Film (FilmDB) menggunakan pendekatan *Polyglot Persistence* dengan dua model *database* yang berbeda:

1) **MongoDB (Document-Oriented Database):** Dipilih untuk menyimpan data dokumen film seperti detail, metadata, dan ulasan pengguna. Keunggulan skema JSON/BSON yang fleksibel memungkinkan penyimpanan tipe data *array* (untuk daftar *cast* dan *genre*) serta dokumen tersarang/*embedded* (untuk ulasan pengguna) dalam satu tempat.
2) **Neo4j (Graph Database):** Dipilih khusus untuk menyimpan data relasi antar entitas. Neo4j menggunakan *Node* (simpul) dan *Relationship* (sisi) untuk memetakan hubungan seperti ACTED_IN, DIRECTED, dan SIMILAR_TO, sehingga proses pencarian rekomendasi film dan analisis jaringan aktor dapat dieksekusi dengan performa tinggi tanpa perlu operasi JOIN yang berat.

Dengan menggabungkan keduanya, kueri filter dasar dapat diproses di MongoDB, lalu hasilnya diteruskan ke Neo4j untuk menemukan *insight* berbasis relasi atau rekomendasi tingkat lanjut.

## 2. SOAL 2: DATA SEDERHANA PADA MASING-MASING DATABASE

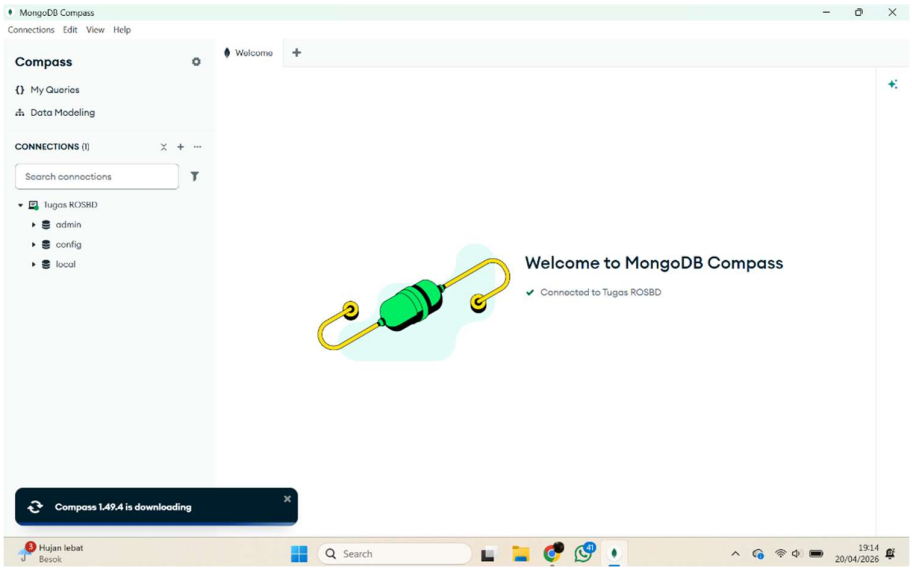
Pada tahap ini, dilakukan proses *data ingestion* (penyisipan data) ke dalam dua *database* yang berbeda secara lokal dan *cloud*.

### 2.1. Ingesti Data ke MongoDB (Document-Oriented)

Proses pembuatan *database* dan *collection* dilakukan menggunakan GUI MongoDB Compass yang terhubung ke *localhost*.

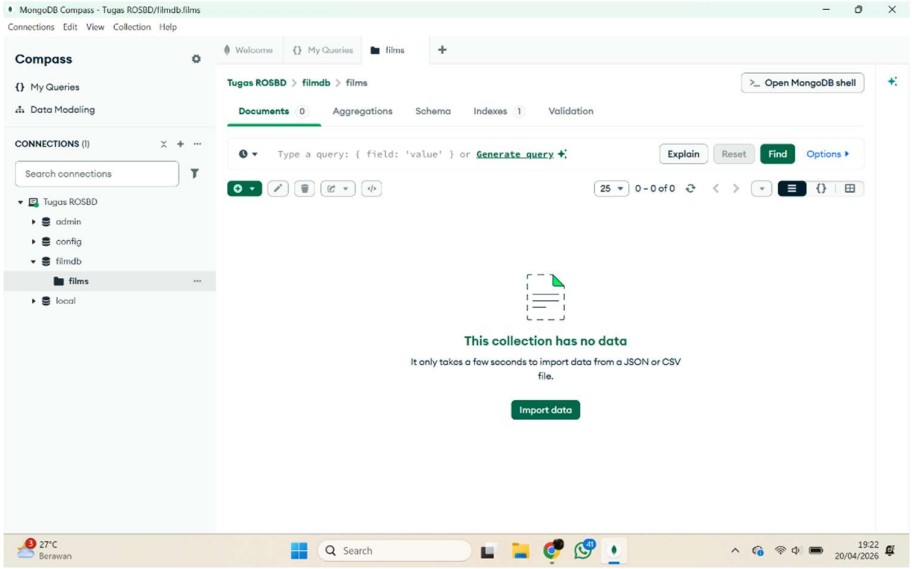
- Langkah 1: Membuat Koneksi dan Database

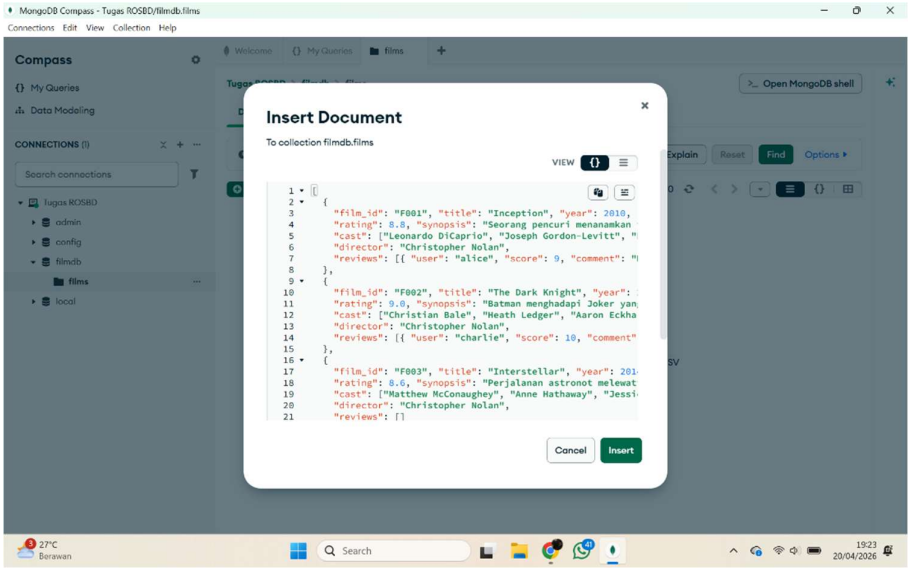
  Menginisialisasi koneksi ke mongodb://localhost:27017 dan membuat *database* baru bernama filmdb dengan *collection* films.




- Langkah 2: Penyisipan Dokumen (Insert Document)

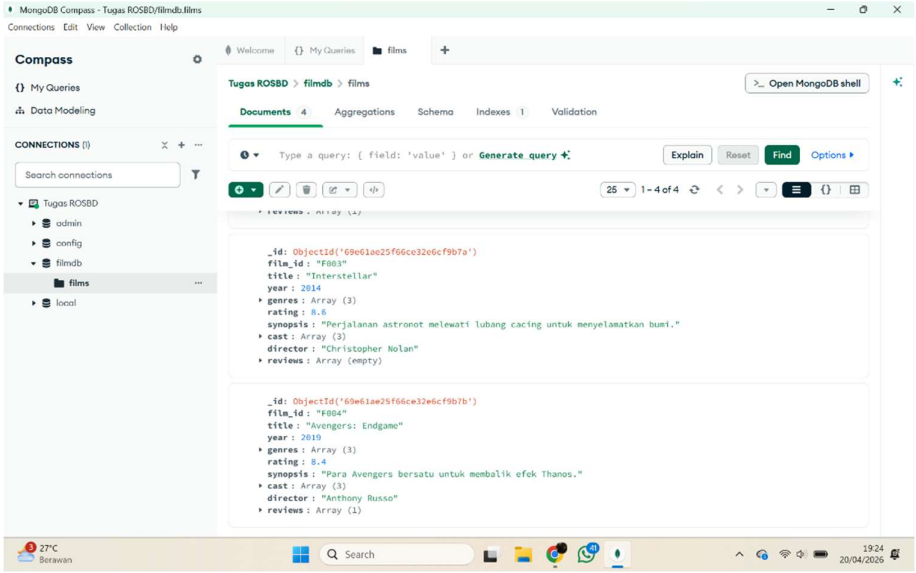
  Memasukkan 4 data film dalam format JSON *Array* ke dalam *collection* films. Format BSON/JSON dipilih agar atribut *array* (seperti genres) dan *embedded document* (seperti reviews) dapat tersimpan dalam satu hierarki tanpa perlu tabel terpisah.




- Langkah 3: Verifikasi Data MongoDB

  Verifikasi bahwa seluruh dokumen telah berhasil tersimpan dan mendapatkan _id (ObjectId) secara otomatis dari sistem.

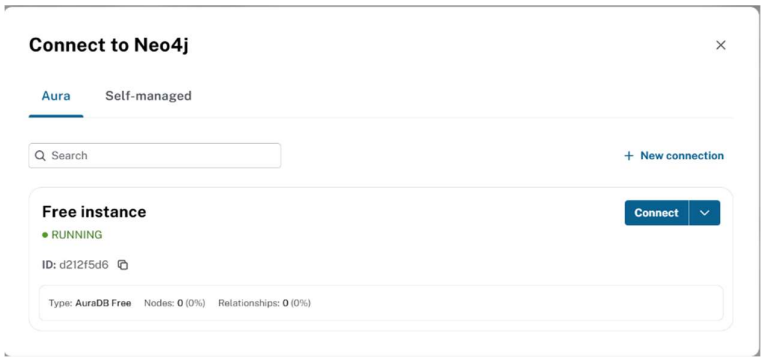



## 2.2. Ingesti Data ke Neo4j (Graph Database)

Penyimpanan data relasi dilakukan menggunakan Neo4j Aura (Cloud) untuk memetakan hubungan antar *Node* (Film, Aktor, Genre) menggunakan *Relationship* (DIRECTED, ACTED_IN, HAS_GENRE, SIMILAR_TO).

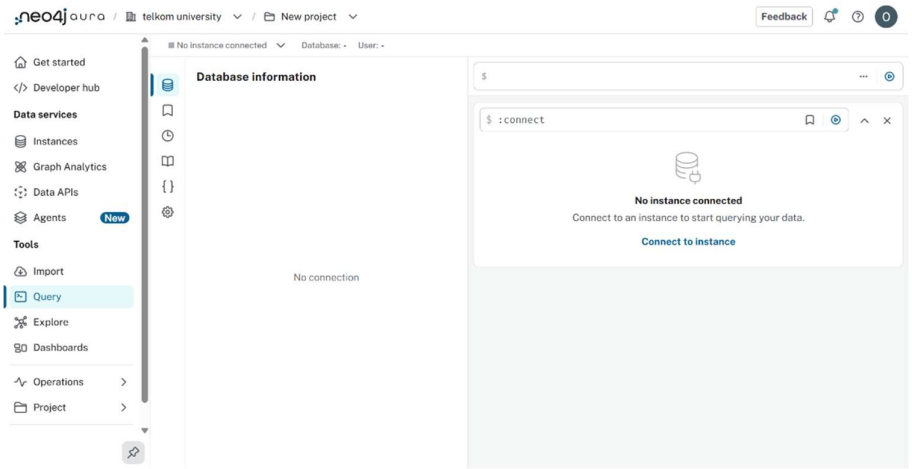
- Langkah 1: Inisialisasi Instance Cloud

  Membuat dan menjalankan *Free Instance* pada ekosistem Neo4j Aura.



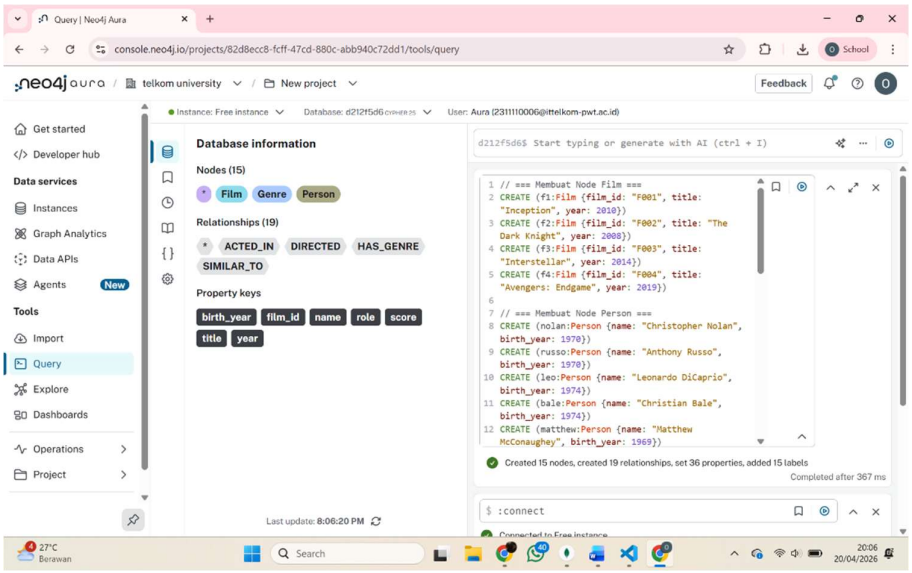

- Langkah 2: Autentikasi dan Koneksi Workspace

  Melakukan *login* ke dalam Neo4j Browser/Query Workspace menggunakan ID Instance dan *password* yang telah digenerate.




- Langkah 3: Eksekusi Query Pembuatan Graph (Cypher)

  Mengeksekusi perintah CREATE Cypher untuk membentuk 15 *Nodes* dan 19 *Relationships* secara serentak.



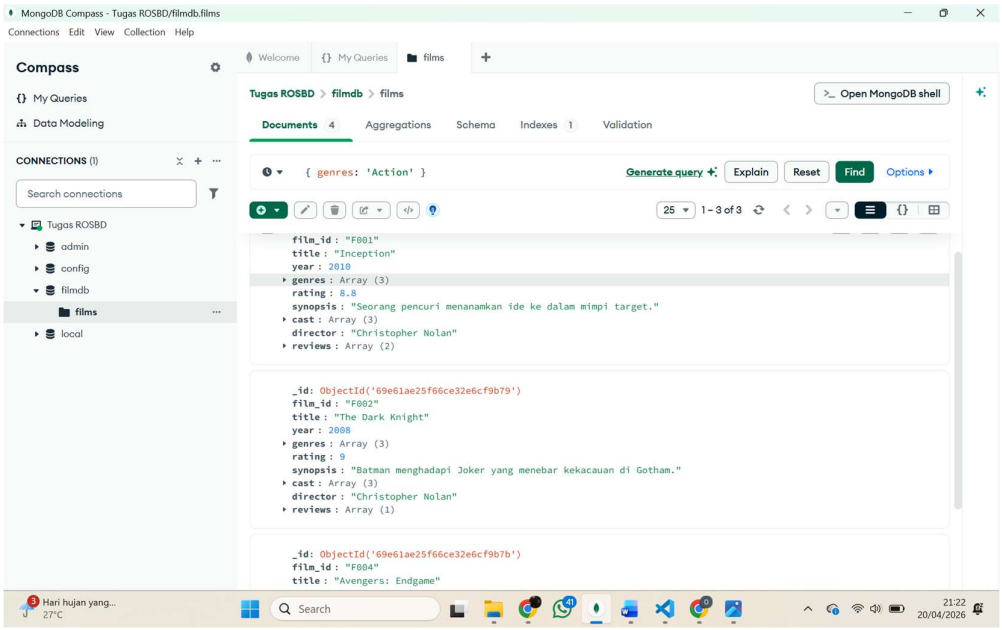

## 3. SOAL 3: QUERY PADA MASING-MASING DATABASE

Setelah data berhasil disisipkan, dilakukan uji coba pemanggilan data (*querying*) secara independen pada masing-masing *database* untuk membuktikan kapabilitas operasionalnya sebelum digabungkan.

### 3.1. Eksekusi Kueri di MongoDB (MQL)

Pada tahap ini, dilakukan pengujian kapabilitas operasional MongoDB dalam melakukan pencarian data menggunakan MongoDB Query Language (MQL) melalui antarmuka grafis MongoDB Compass.

- Tujuan: Menguji efisiensi pengambilan dokumen spesifik berdasarkan kriteria tertentu di dalam koleksi films.
- Kriteria Filter: Memfilter seluruh basis data film untuk hanya menampilkan judul yang memiliki genre "Action".
- Langkah Praktikum:
  - Membuka *collection* films di dalam *database* filmdb pada MongoDB Compass.
  - Pada bar pencarian (Filter), dimasukkan kueri: { genres: 'Action' }.
  - Mengklik tombol Find untuk mengeksekusi pencarian.




Sistem berhasil melakukan pemindaian dokumen dan mengembalikan hasil berupa film-film yang relevan, seperti *Inception* (F001) dan *The Dark Knight* (F002), dan Avengers (F004).

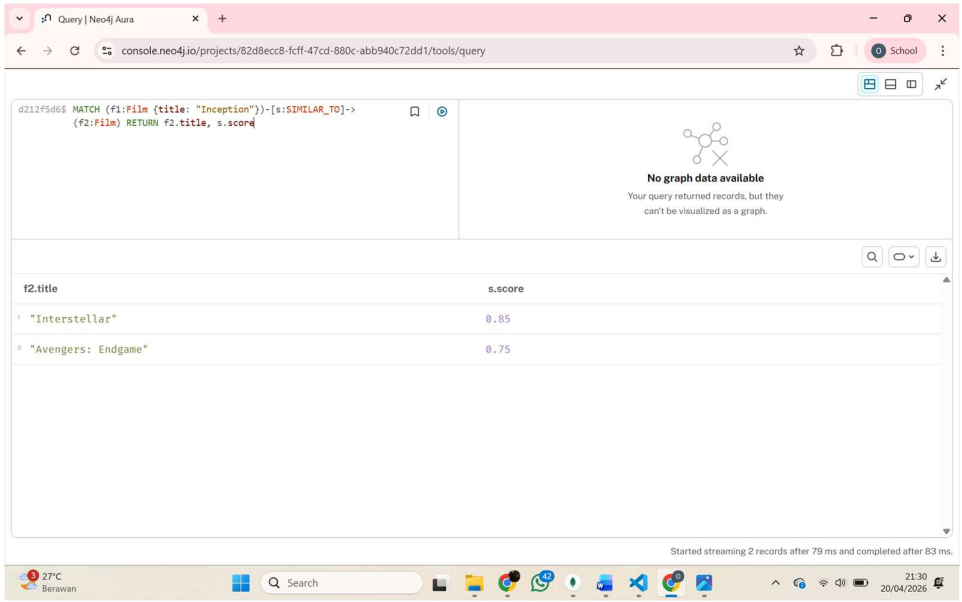
### 3.2. Eksekusi Kueri di Neo4j (Cypher)

Tahap selanjutnya adalah pengujian fungsionalitas pada Neo4j Aura (Cloud) menggunakan bahasa kueri Cypher. Fokus utama pada tahap ini adalah membuktikan efisiensi pencarian relasi antar data.

- Deskripsi: Menguji mekanisme Pattern Matching untuk mencari entitas film yang memiliki tingkat kemiripan (*similarity*) dengan film "Inception". Operasi ini memanfaatkan keunggulan Index-Free Adjacency, di mana *database* menelusuri

relasi melalui *pointer* fisik antar *node* tanpa perlu melakukan proses *join* tabel yang intensif secara komputasi.

- Langkah Praktikum:
  - Membuka *Workspace* kueri pada Neo4j Aura yang telah berstatus *Running*.
  - Menghapus seluruh kueri sebelumnya pada editor.
  - Memasukkan perintah Cypher berikut: MATCH (f1:Film {title: "Inception"})-[s:SIMILAR_TO]->(f2:Film) RETURN f2.title, s.score
  - Mengklik tombol Play untuk mengeksekusi kueri.




Sistem secara instan menemukan pola relasi SIMILAR_TO yang keluar dari *node* Inception. Output menampilkan dua film rekomendasi, yaitu Interstellar dengan skor kemiripan 0.85 dan Avengers: Endgame dengan skor 0.75. Hasil dapat ditampilkan dalam bentuk Table (tabulasi data) maupun Graph (visualisasi gelembung entitas dan garis relasi). Masukkan SS yang menampilkan hasil dari *query* tersebut (biasanya akan muncul tabel atau gelembung graf yang menunjukkan Interstellar dan Avengers).

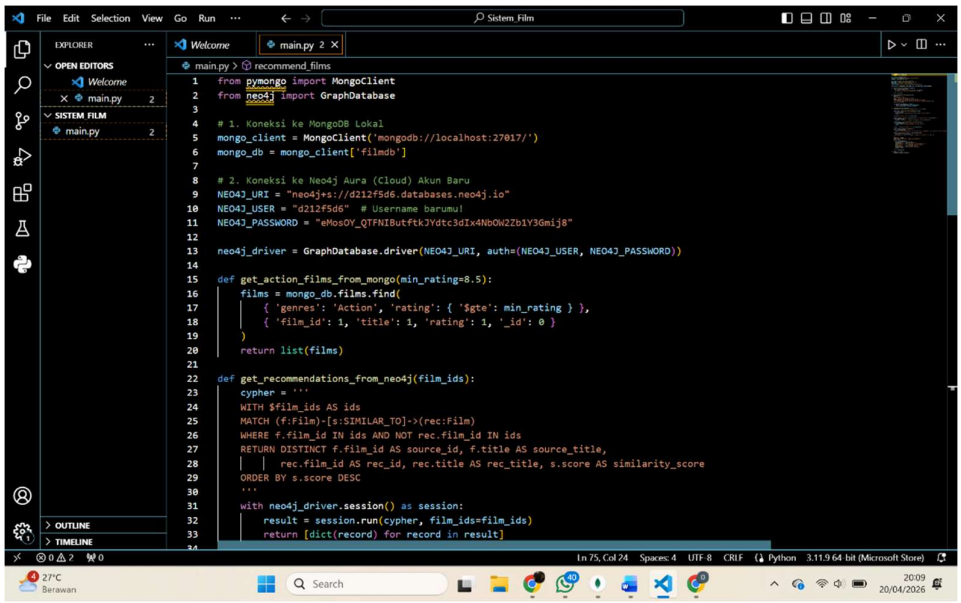
### 3.3. Eksekusi Kueri di MongoDB (MQL)

Tahap final dari Soal 3 adalah menghubungkan kedua database menggunakan *middleware* Python untuk menjalankan skenario sistem rekomendasi film secara otomatis.

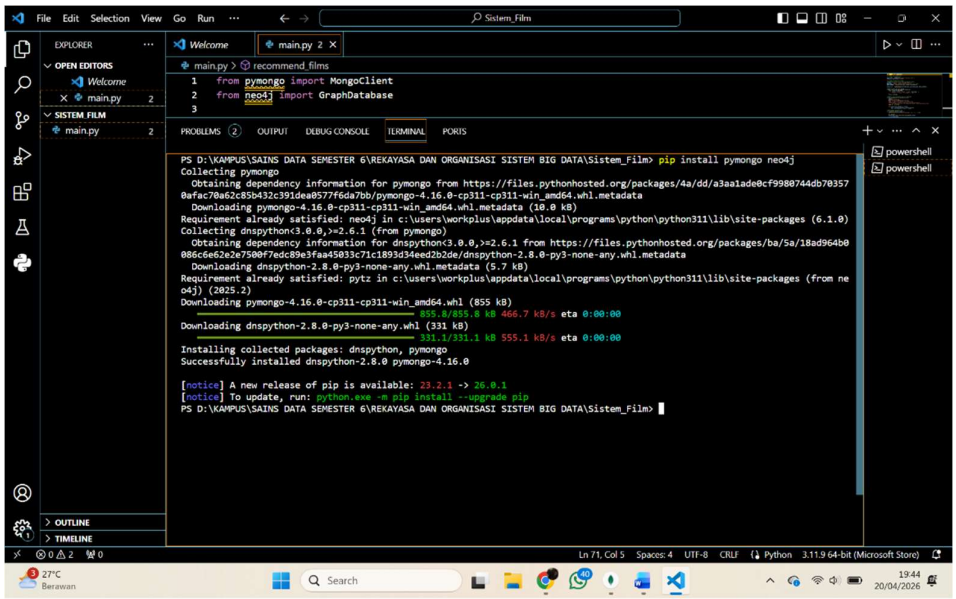
- Deskripsi: Proses integrasi ini menggunakan Python untuk menjembatani perbedaan model data antara MongoDB dan Neo4j. Sistem mengambil data awal dari MongoDB (berdasarkan filter tertentu), mengekstrak kunci tautan (film_id), dan mengirimkannya ke Neo4j untuk diproses melalui *Graph Traversal*.

- Langkah Praktikum:

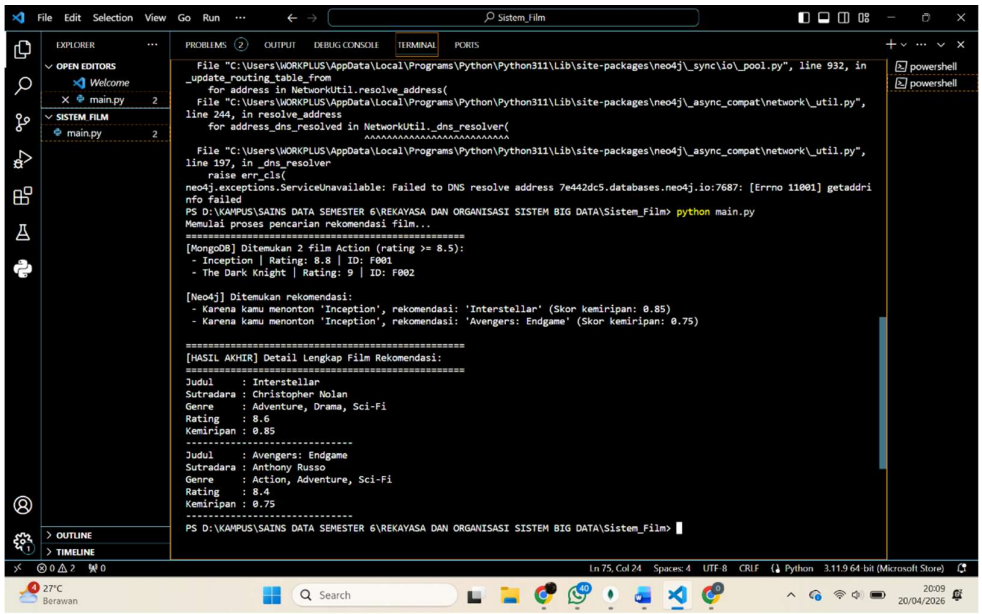
  - Membuka folder proyek Sistem_Film pada **VS Code** dan menyiapkan file main.py.




  - Menginstal *library* pendukung melalui terminal VS Code dengan perintah: pip install pymongo neo4j.



- Menjalankan skrip Python yang telah mengintegrasikan URI koneksi, *username*, dan *password* dari Neo4j Aura Cloud serta koneksi *localhost* MongoDB.




Program sukses melakukan filter pada MongoDB untuk film genre "Action" dengan rating ≥ 8.5. ID film hasil filter diteruskan ke Neo4j untuk melakukan penelusuran relasi SIMILAR_TO. Sistem berhasil menarik kembali data detail (Sutradara, Genre lengkap, dan Rating) dari MongoDB berdasarkan rekomendasi yang ditemukan oleh Neo4j.

## 4. SOAL 4: ALUR PROSES QUERY

## 5. SOAL 5: ANALISIS PERBEDAAN DAN KELEBIHAN MASING-MASING DATABASE

a. Perbedaan

| Karakteristik | MongoDB | Neo4j |
|---|---|---|
| Model Database | Document-Oriented Database | Graph Database |
| Struktur Penyimpanan | Dokumen dalam format BSON/JSON (mendukung hierarki seperti array dan embedded document). | Node (simpul) dan Relationship (sisi). |

| | | |
|---|---|---|
| Fokus Operasional | Penyimpanan data entitas yang kaya akan atribut dan metadata. | Penyimpanan data relasi antar entitas yang kompleks. |
| Bahasa Query | MongoDB Query Language (MQL) | Cypher |
| Peran dalam Sistem Film | Menyimpan detail lengkap film, metadata (genre, rating, dll), daftar cast, dan ulasan pengguna. | Memetakan hubungan seperti ACTED_IN, DIRECTED, SIMILAR_TO, dan HAS_GENRE untuk analisis dan rekomendasi. |

b. Kelebihan MongoDB dalam sistem film

- Fleksibilitas Skema Data

  Format BSON/JSON memungkinkan penyimpanan struktur data yang kompleks dalam satu tempat. Daftar cast dan genre dapat disimpan sebagai tipe data array, sementara ulasan pengguna dapat disimpan sebagai dokumen tersarang (embedded document) tanpa perlu memecahnya ke dalam tabel-tabel terpisah seperti pada basis data relasional.

- Efisiensi Filtering Dasar

  MongoDB sangat optimal untuk memproses kueri filter dasar (seperti pencarian genre "Action" dengan rating tertentu) dan langsung menyajikan keseluruhan kelengkapan dokumen data film tersebut

c. Kelebihan Neo4j dalam sistem film

- Performa Pemrosesan Relasi yang Tinggi

  Neo4j secara spesifik dirancang untuk menavigasi jaringan entitas. Pencarian rekomendasi film dan analisis jaringan aktor dapat dieksekusi dengan performa tinggi tanpa perlu melakukan operasi JOIN yang berat secara komputasi.

- Pencarian Pola (Pattern Matching) Cepat

  Neo4j memanfaatkan Index-Free Adjacency, di mana sistem menelusuri relasi langsung melalui pointer fisik antar node. Hal ini terbukti efisien saat sistem secara instan menemukan pola relasi SIMILAR_TO untuk memberikan skor rekomendasi film.